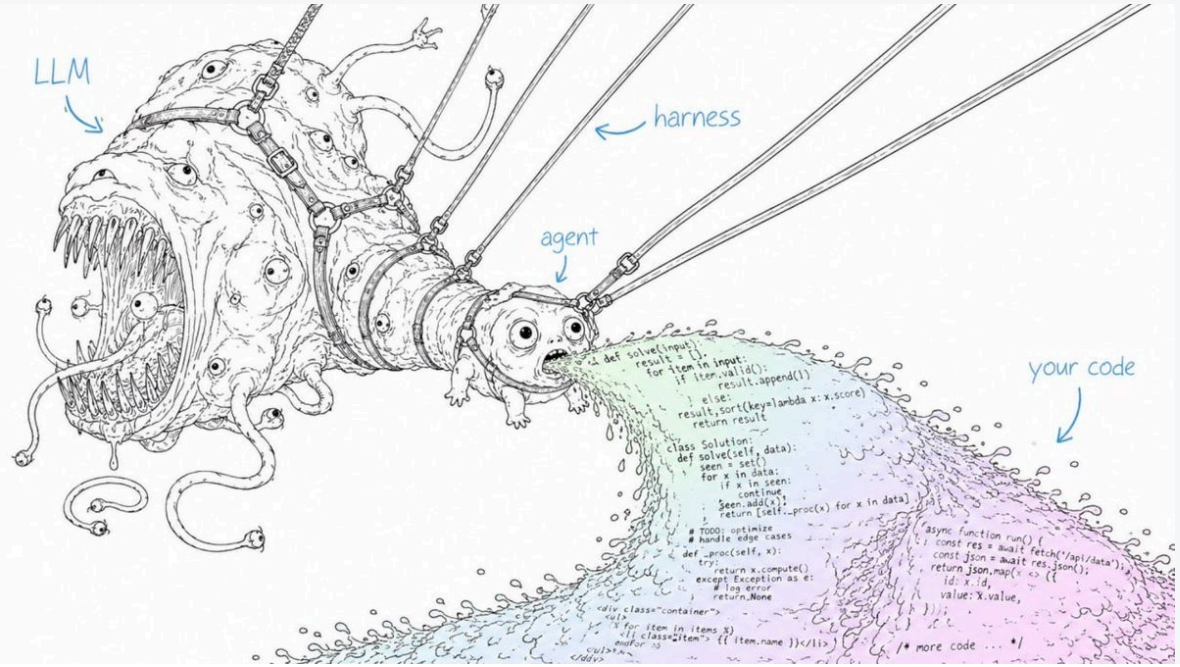
Agentic AI Workflows & Harness Engineering

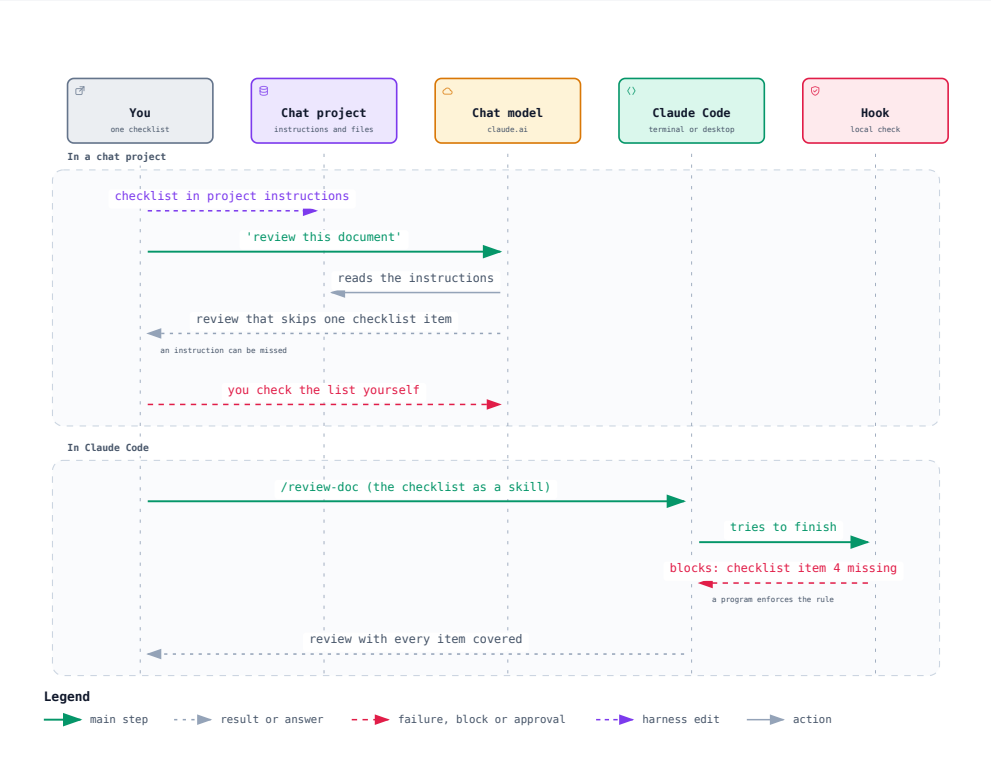
Fundamentals & How to Build Your Own

Sergei Wallace



Audience and scope

For anyone who uses AI models for work, from coding agents to chat apps



- Who it is for
  - Engineers running coding agents
  - Other technical workers
  - People who only use a chat app
- Main tools: Claude Code first, then Codex
  - Most ideas carry over to the chat and desktop apps
- My own harness is one example
  - Build from the problems you hit; a copied harness brings its author's habits

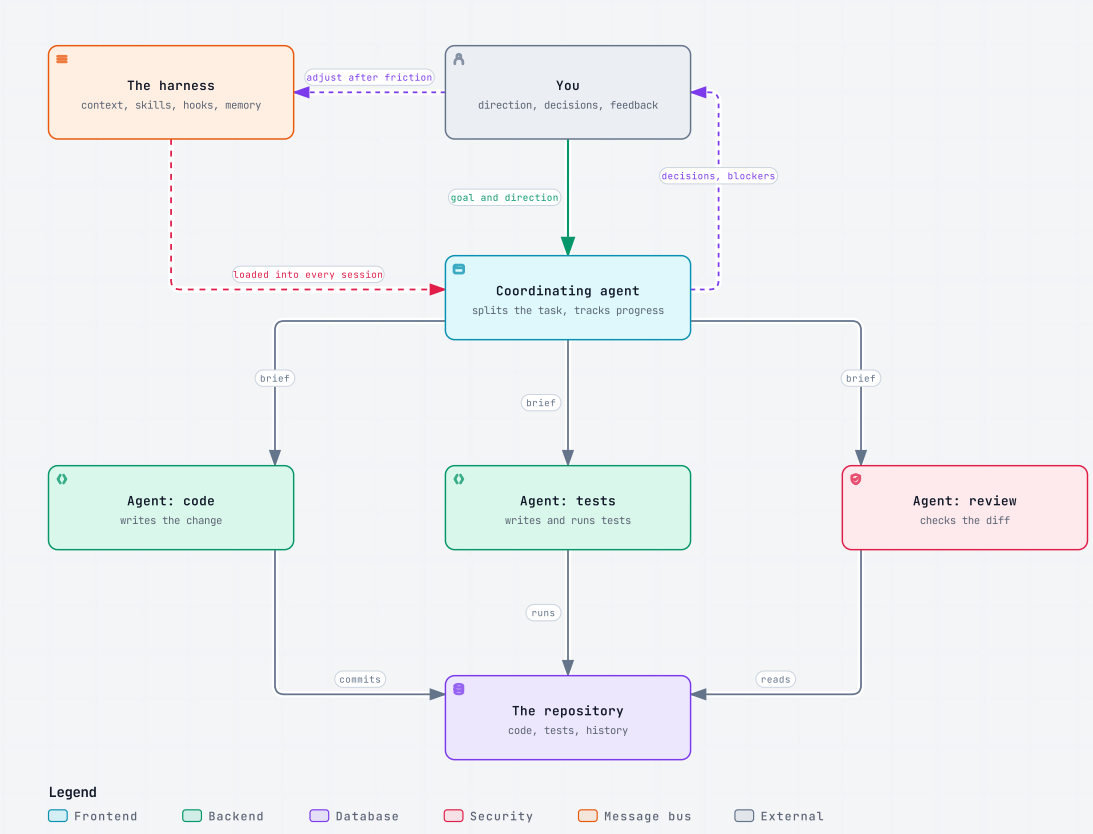
The conductor model

> “the AI engineer conducts an AI fleet using orchestrated workflows”
>
> The talk's meeting description

- You direct agents through workflows
  - A goal, a brief and the checks the result must pass
- You adjust the harness so they need less direction
  - A repeated correction becomes a workflow, hook or test
- Advanced: agents coordinate on long tasks
  - You step in on decisions, blockers and feedback

Talk route

Every topic has its own pages on the site.



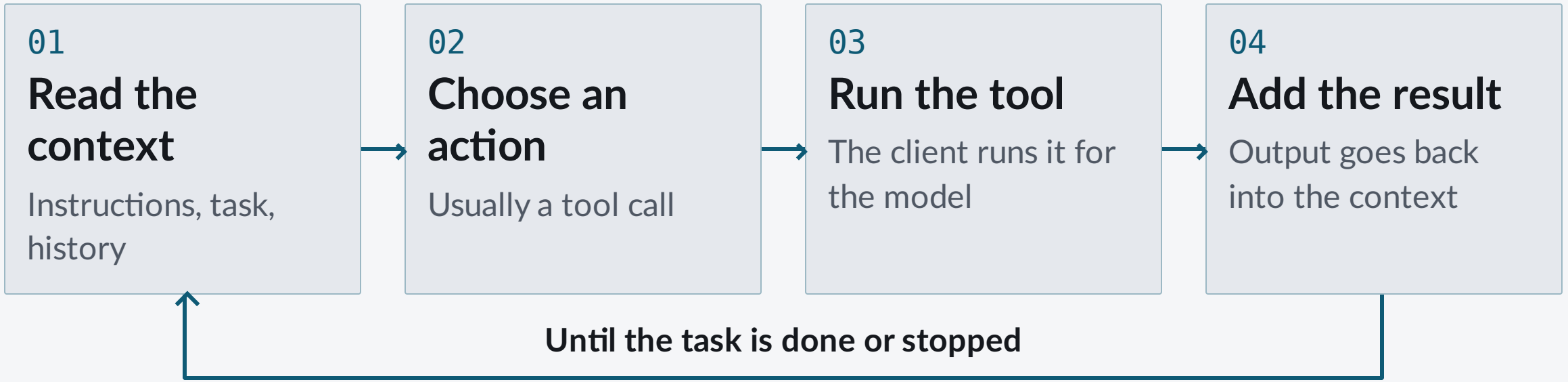
01 Agentic AI and harnesses

02 Building blocks in Claude Code and Codex

03 How the blocks combine

04 Building your own harness

The agent loop



- A chat model answers one prompt; an agent repeats this loop toward a goal
  - Tools let it act: read files, run commands, search, call services
  - Every part of a harness acts on one step of this loop

Context window and sessions

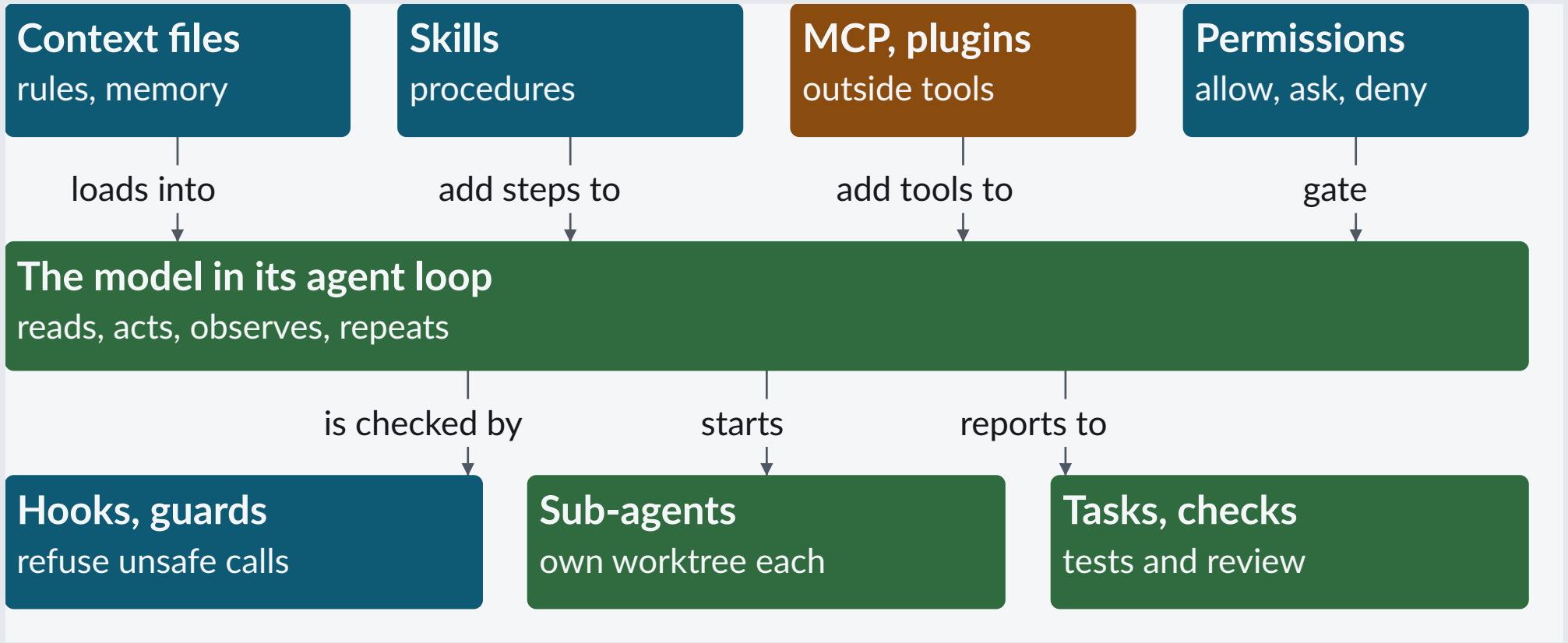
- Everything the model sees on one turn
  - Instructions, the conversation, tool results
- Nothing carries between sessions unless written to a file
  - The harness loads those files again next session
- When an agent ignores a rule, check its context
  - It never saw the rule, or it saw too much else

1M

tokens in the context window of several current Claude models

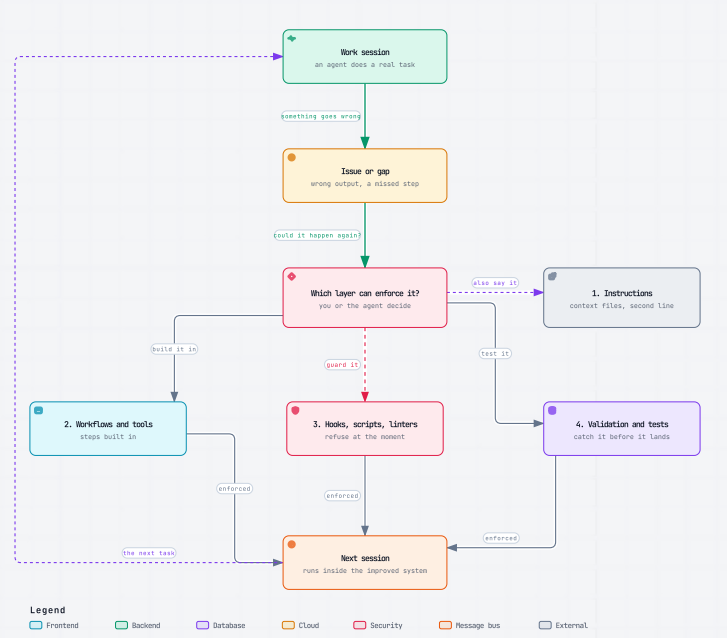
Parts of a harness

"Everything in an AI agent except the model itself" (Birgitta Böckeler)



A systems approach to harness engineering

Each fix goes into the system; layers 2 to 4 enforce it.
Typed or prompted it twice? Change the harness, sooner if a repeat is costly.



01 Instructions: context files, skills and prompts

02 Workflows and tools that build the steps in

03 Hooks, scripts and linters: enforce how you want it, catch mistakes

04 Validation and tests before a change lands

Harness cost and effect

- Each part of a harness costs tokens on every turn
  - Context files, skill and agent descriptions, MCP tool lists
- Each part has to justify its cost
  - Measure before adding; remove what does not pay

Task success: moved about 2 to 5 points across 21 model and harness pairs

Cost: Claude Code about twice a minimal four-tool harness on SWE-bench Lite

The building blocks

BLOCK	WHAT IT IS	MAIN USE
Context files	Instructions the client loads	Guide how the model works
Skills and plugins	Instructions that load on use	Hold reusable procedures and workflows
Hooks and checks	Code at fixed points	Enforce how you want the output to be, and stop the mistakes you have seen
Templates and briefs	Fixed shapes for prompts	Consistent hand-offs
Sub-agents	Separate contexts	Delegated and parallel work

Claude Code and Codex

Claude Code

- `CLAUDE.md` context files
- Skills; sub-agents in `.claude/agents/`
- Hooks at 33 events
- Goal, loop and compact commands
- Terminal, editors, desktop and web

Codex

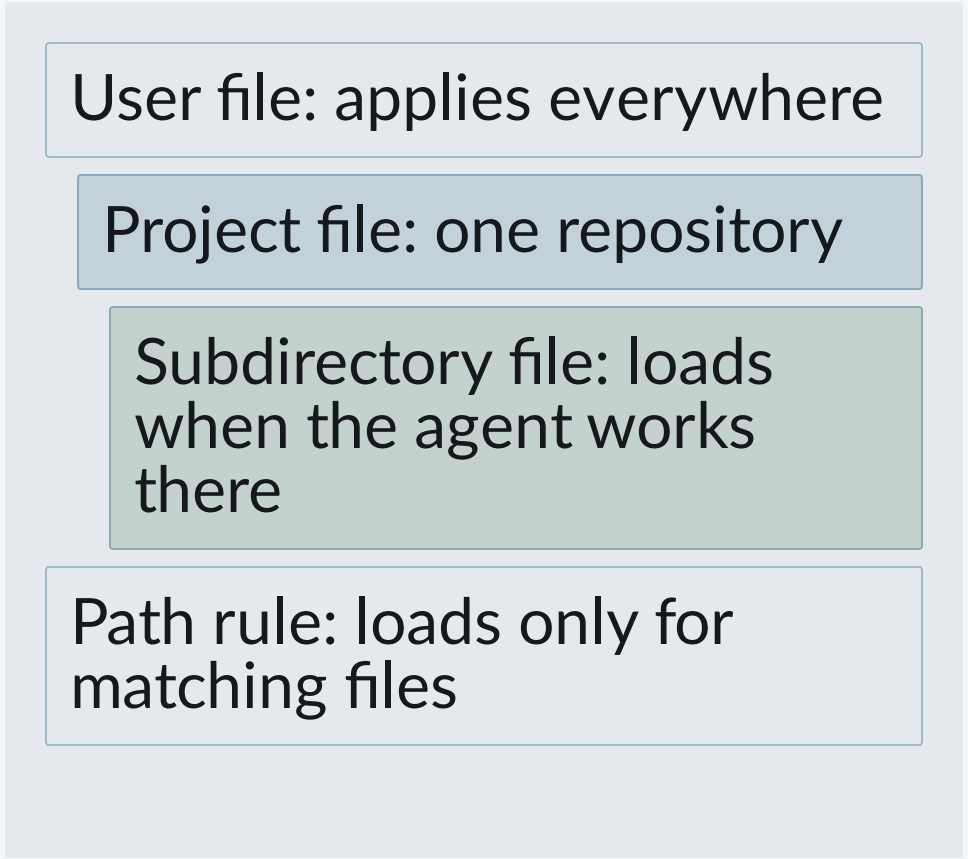
- `AGENTS.md` context files
- Skills; sub-agents as TOML in `.codex/agents/`
- Hooks at 14 events
- Goal and compact commands; loop not available
- Imports a Claude Code or Cursor setup

Native commands

COMMAND	WHAT IT DOES
/goal	Keep working until a condition is met
/loop	Repeat a prompt on an interval
/compact	Summarise to free context, with a focus
/rc	Control a session from another device; needs a claude.ai sign-in
/context	Show what fills the context window, by category

Context files in practice

- `CLAUDE.md` for Claude Code, `AGENTS.md` for Codex
  - Loaded at launch from the working directory and above
- Path rules in `.claude/rules` load for matching files only
  - A `paths` list in the rule's frontmatter
- Keep the always-loaded file short
  - Under about 200 lines
  - One line per rule, linking to a procedure (hub and spoke)



Skills and plugins

Skills

- A folder of instructions that loads only when used
  - Its short description is always in context
- The model can start some; only the person can start others
  - Set in the skill's frontmatter
- Person-only for side effects
  - Publishing, sending, merging
- Agent Skills open standard; Codex uses it too

Plugins

- Skills, hooks, agents and MCP servers as one unit
  - Installs and shares as one package
- “Plugin module”: my name for one sharing a code library
- Each enabled plugin costs context on every turn
  - Its skill and agent descriptions stay loaded
- A plugin with a hooks module is a mod

Templates and briefs

- Agents write most of a harness's prompts
  - A lead agent writes each sub-agent's brief
  - A stub is a fill-in-the-blanks template
- A sub-agent sees only its brief
  - Its parent's rules reach it only through the template
- The harness can send text into a live session
  - A reminder after compaction
  - A wake-up for an idle session
  - A message from another session
- To start: one template for the task you delegate most

Enforcing rules in code

- Enforce as much as possible in code
  - Instructions only where no program can check
- Written instructions are followed less consistently over time
  - As the context fills, as tasks pile up, after a task switch
- Claude Code also runs function hooks inside the client
- These run the same way every time:

Hook

Script or linter

Test

CI gate

Enforcement options

01
Context rule
Preferences and judgment calls

→

02
Path rule
Rules for some files only

→

03
Skill
Fixed steps, when it runs

→

04
Hook or guard
Checked when the agent acts

→

05
Test or CI gate
Checked at commit, push or merge

- Put each rule in the strongest step that can check it
  - Keep a context rule only for what no program can check
- `/hooks` shows every configured hook and where it comes from
  - Read-only

Process-kill guard case study

01
Rule in the brief
No kills by name pattern

→

02
An agent breaks it
Three unrelated test runs stop

→

03
A hook enforces it
Refused before it runs

→

04
Refusal says the fix
One process id or group

Hook refusal

```
$ pkill -f "pytest tests"
refusing to kill processes by command-line pattern. ...
Safe forms: kill a PID you captured yourself (...);
kill your own process group (`kill -- -<pgid>`, ...)
```

Cost of guards

- A check and a risky step

```
check          # fails
step           # runs anyway

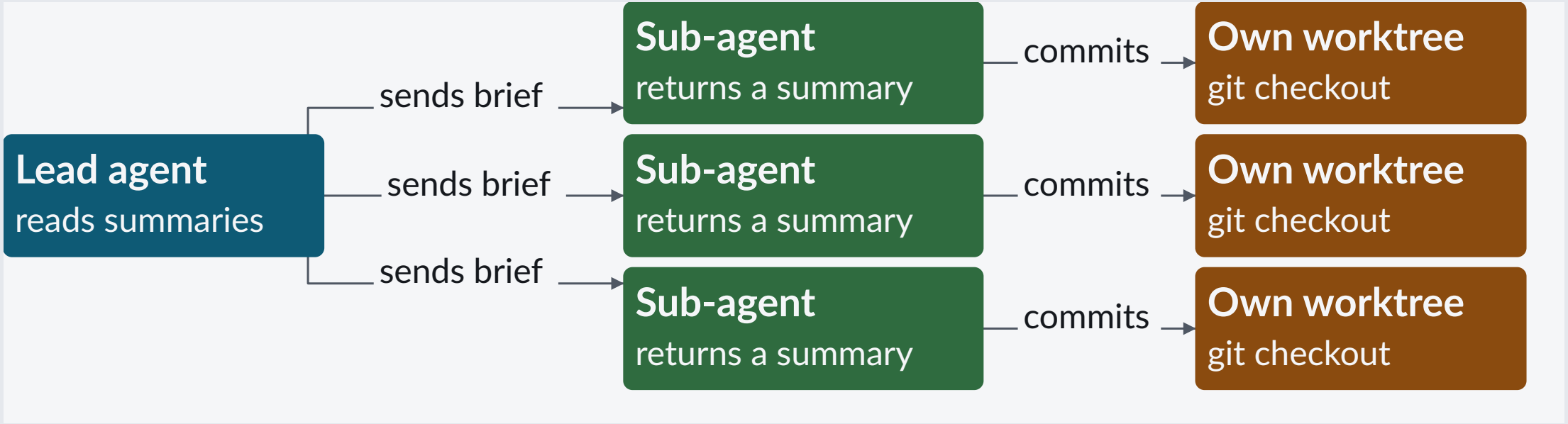
check && step
# step runs only if
# check passes
```

- A guard refuses what it cannot prove safe
  - So it blocks some legitimate work
- Guards need upkeep
  - And add time to every call they check
- A hung hook can block all work
  - Give each a time limit and a visible failure
- Gate the step on the check
  - Two lines let a failed check through

Sub-agents and agent profiles

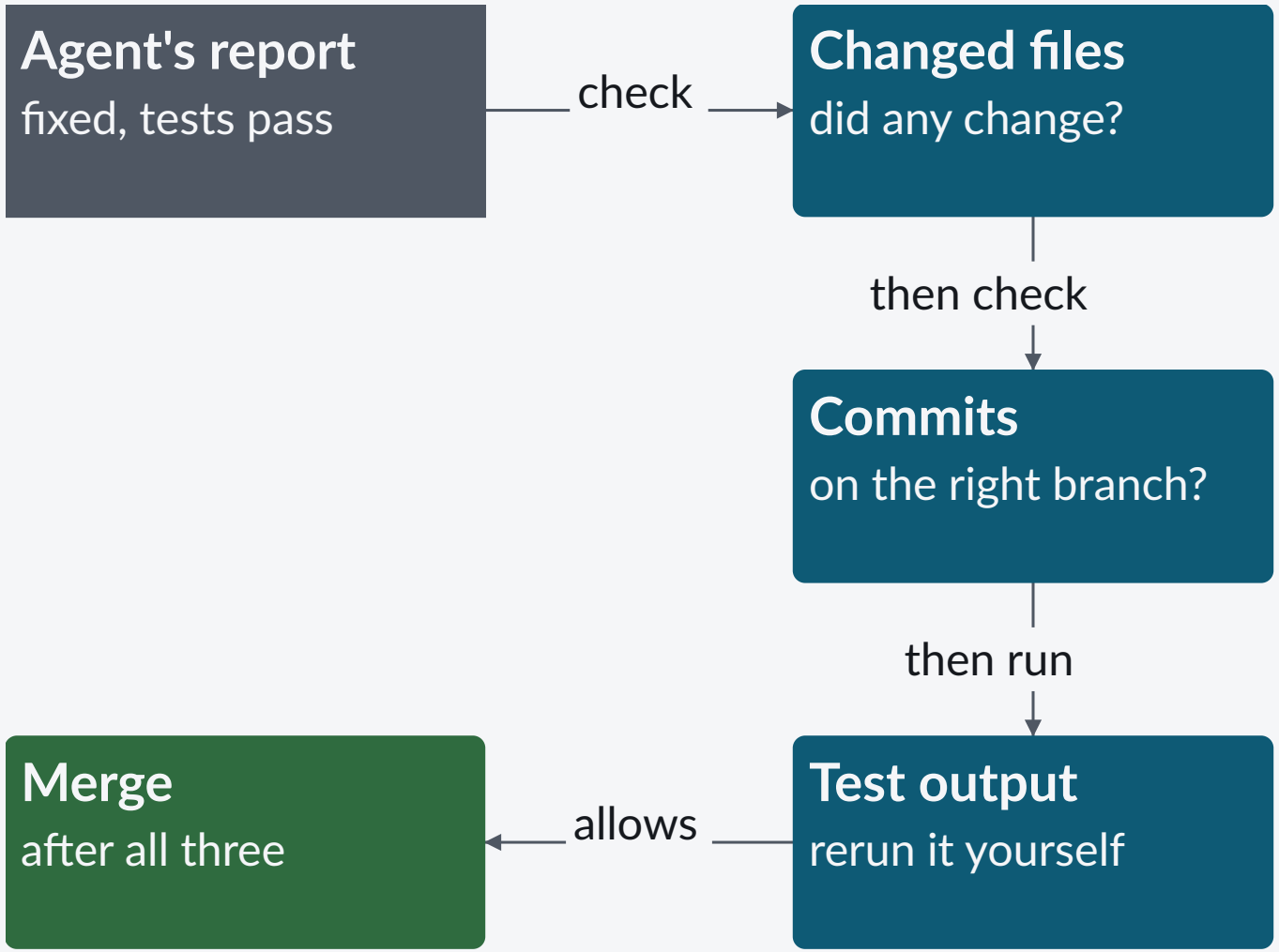
- A sub-agent works in its own context
  - It returns a summary, so the lead's context stays small
- An agent profile pins a role's tools, model and permissions
  - `.claude/agents/` in Claude Code; TOML in `.codex/agents/` in Codex
- Pick the model per task
  - The strongest for decisions and synthesis
  - Cheaper ones for bulk reading and mechanical edits
- A brief states the goal, inputs, return format and limits
  - To start: one read-only researcher profile and one brief template

Delegation with worktrees



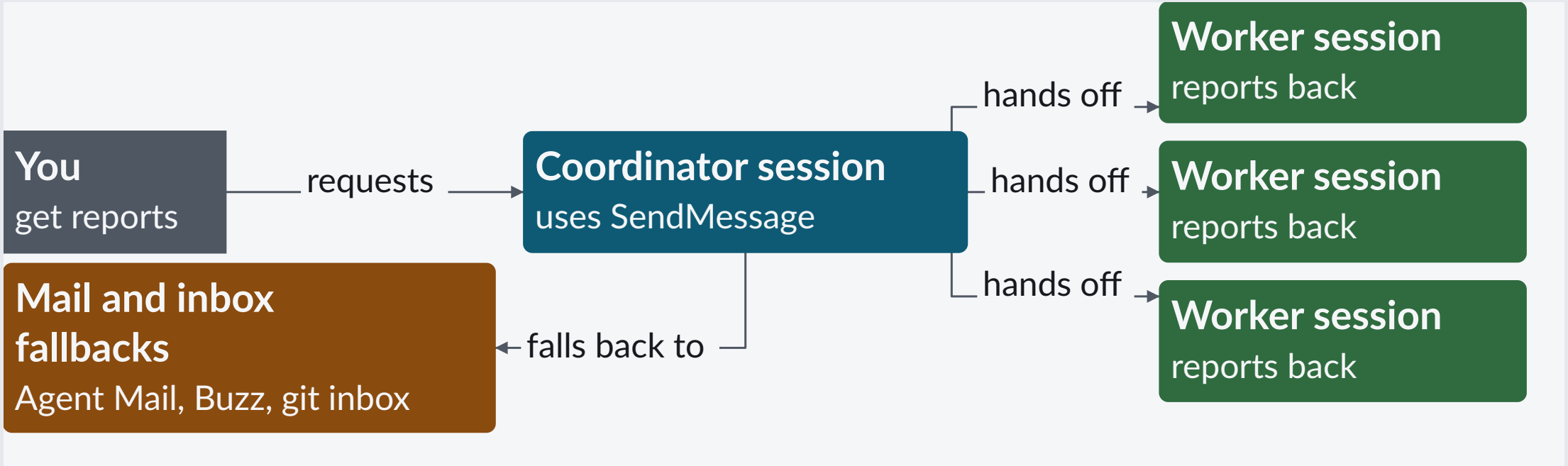
- Give each writing agent its own worktree; bound how many agents run at once and how deep they nest
- `/tasks` lists background work, finished sub-agents included
- `/subtask` starts a sub-agent that inherits the conversation; `/fork` copies the session to the background

Checking delegated work



- Agents can report work they did not do
  - A fix report with passing tests, and no file changed
- Read key passages, not only the summary
  - A summary can drop the caveat that matters
- Isolation can fail
  - Check which branch the commits landed on

A coordinator and worker sessions



- A coordinator session gives you fewer places to deal with
- `ListAgents` and `/list-agents` show who a session can message

Two kinds of orchestration

Prompt-driven

- A lead agent directs others through briefs and profiles
  - It writes briefs, launches sub-agents and joins their results
- Fits work whose steps depend on judgment
  - The steps change as the work goes

Programmatic

- A script or graph holds the plan
  - Dynamic workflows in Claude Code; LangGraph
  - The plan does not take up Claude's context
- Fits known steps, parallel agents, resuming after failure
  - By default 16 agents at once and 1,000 in total
- `/workflows` watches, pauses and saves dynamic workflows

Goals and loops

- Claude Code

```
/goal the test run prints
  no failures

/loop 5m check whether the
  deploy finished
```

- `/goal` runs until a small model judges it met
  - It reads the conversation each turn
- Name printed evidence in the condition
  - Such as a test summary line
- `/loop` repeats a prompt on an interval
- Give each loop an exit and a limit
  - Two or three automatic continuations

Context window management

01
The context fills
Mistakes become more likely

→

02
Write state to files
Task, decisions, next step

→

03
Compact
A summary replaces the history

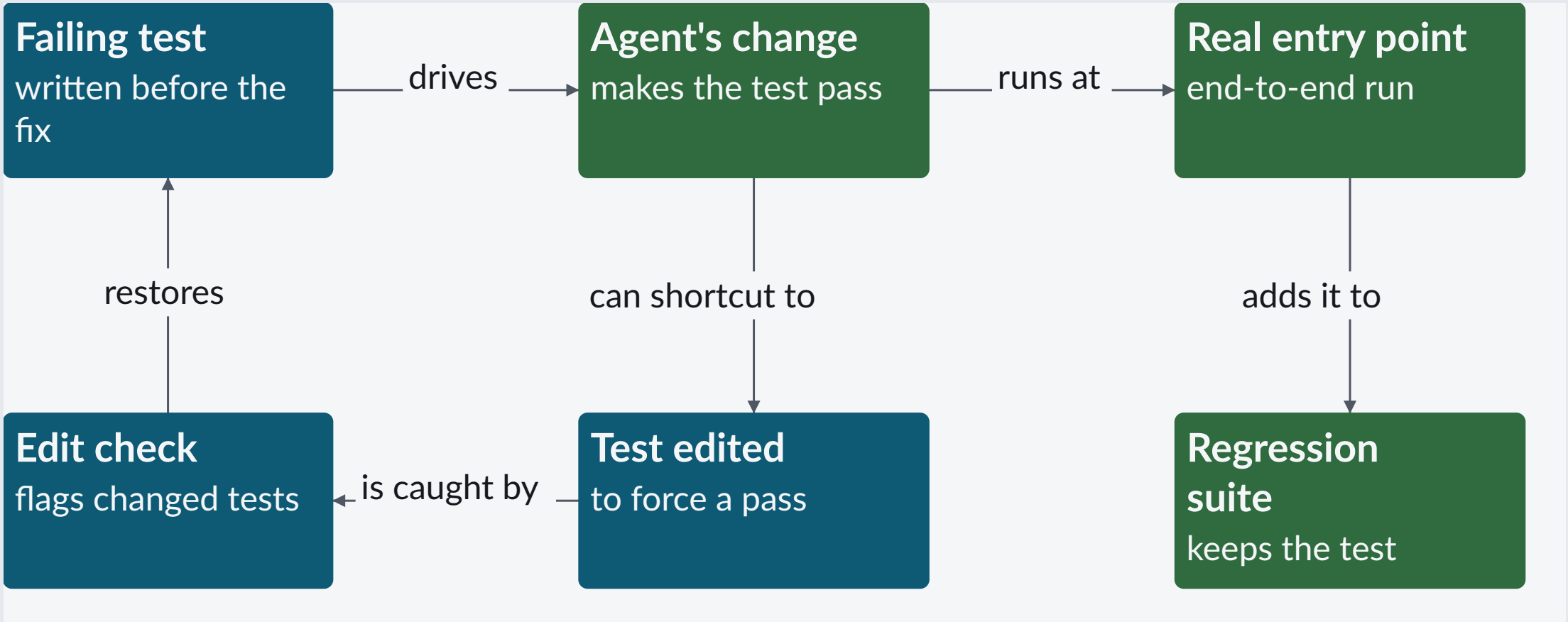
→

04
Resume from the files
A tracker holds the real task list

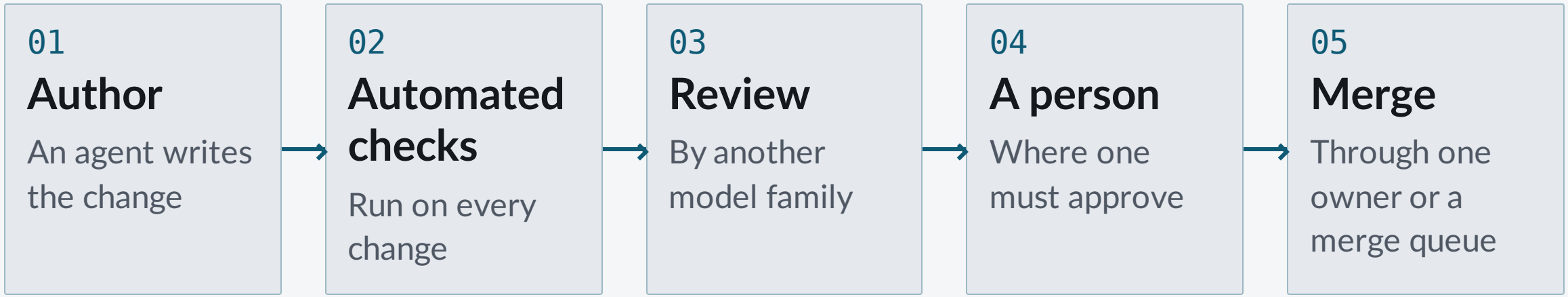
I compact at about 50 to 60 percent; the right point changes as models improve.

`/context` shows what is filling the context window, by category, with suggestions.

The test-first loop



Path of one change



- At high volume nobody reads every change
  - Automated checks and agent review decide most of what lands
- Have a separate agent verify each review finding
  - A reviewer asked to find problems usually reports some

Review practices

- A reviewer from another model family
  - It does not share the author's blind spots
- Fix the acceptance criteria before the review starts
  - Stop when the findings stop being new
- Cap the rounds; re-review only the new diff
  - Claude Code's team: one-round pass rate rose from 65% to 78%
- An agent's approval does not replace a person's where one must approve
- `/code-review` (alias `/review`) and `/security-review` run in a separate context

Open-ended review

> “A reviewer that’s asked to find problems will always find something. If you fix every finding, you end up with extra abstraction layers and tests for cases that can’t happen.”
>
> Claude Code best practices

- Say in the brief that no change is a valid result
  - Pair it with the request to look for improvements
- I see it lead to overly defensive code
  - Layers and tests for cases that cannot happen

Human gates

Asking the person too often is also a failure: each question costs their time and stops the work.

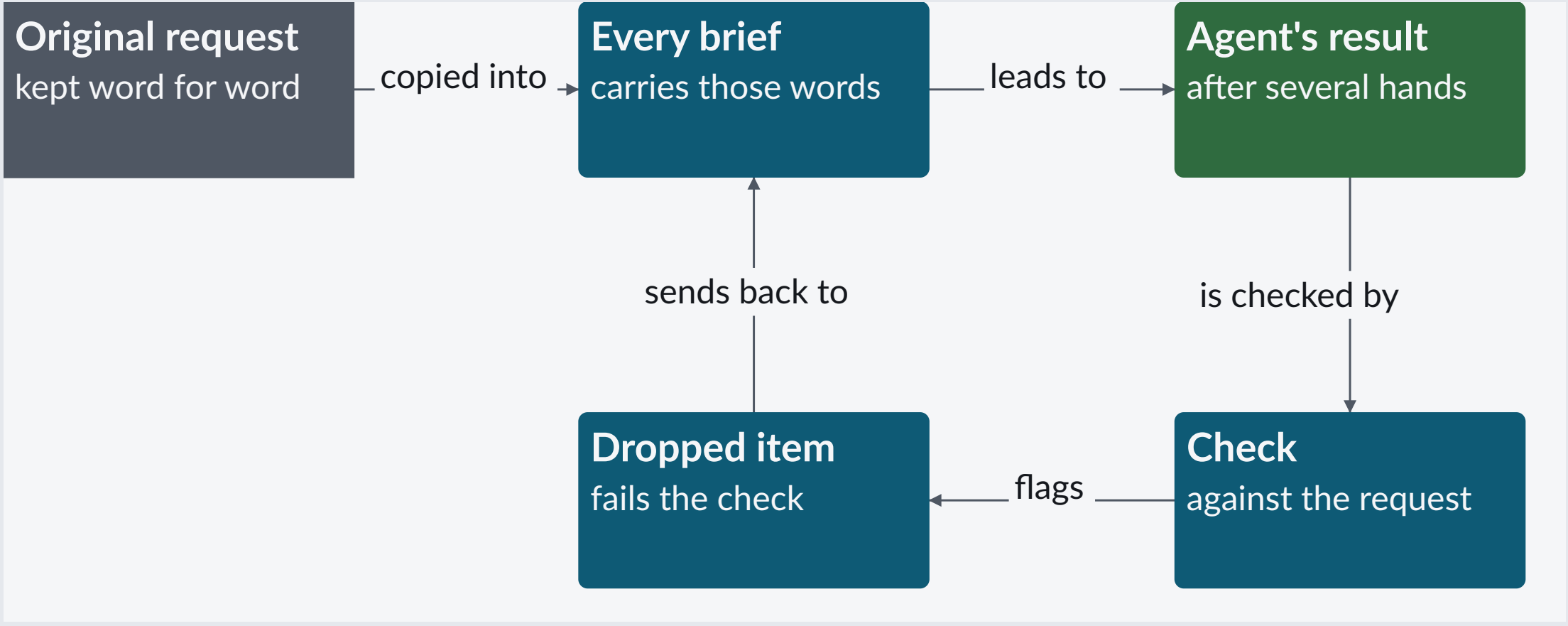
Needs a person

- Publishing, sending and deleting
- Spending money
- Architectural choices
- Write the list first
  - The actions an agent must never take alone

The agent decides and records why

- Contained technical calls
  - Each recorded with its reason
- `/permissions` sets allow, ask and deny rules
- Auto mode: a classifier approves routine actions
  - For long runs

Alignment with the original request



A blocked agent investigates first, and asks only if a gate still applies

Evaluating harness changes

> “When we evaluate ‘an agent,’ we’re evaluating the harness and the model working together.”
>
> Anthropic, Demystifying evals for AI agents, 2026

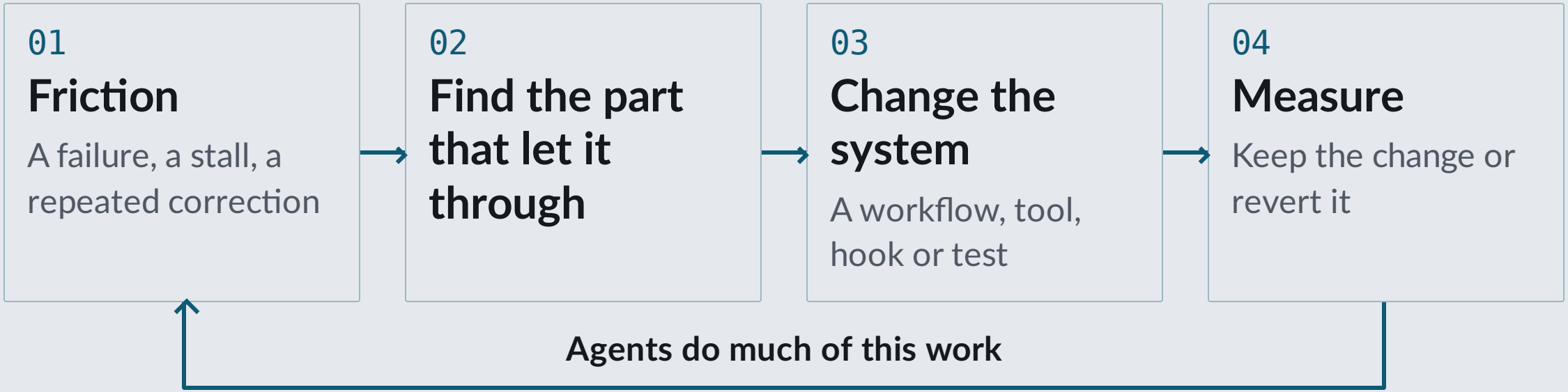
- A harness change needs the same evaluation as a model change
  - Without a baseline you cannot tell whether it helped
- Fix the model and the task set; change one thing
  - Run several times and compare before and after

Logging and evaluation

- Log commands run, files changed, time spent and cost
  - Put a budget check before any job that makes paid model calls
- Measure the harness itself
  - Most of the time to merge can be the wait for the last test run
- `/insights` reports on your recent sessions
  - Friction points included

- One change at a time, fixed task set, before and after
  - Run several times; compare the spread
- Start with 20 to 50 real-failure tasks (Anthropic)
  - Keep capability evals apart from regression evals
- To start: ten tasks from your own past failures
  - Run them before and after one harness change

The improvement loop



- Keep a list of friction
  - Each failure, stall, or correction needed twice
- A context-file line can back the fix up
  - On its own it is applied inconsistently

Agents improving the harness

- An agent can file the bug, write the fix and add the check
  - You review the change, not every step
- Skills and templates turn long instructions into short ones
  - Agents write most of the prompts in a large harness
- Keep a person in the loop where it matters
  - Outward, destructive and architectural decisions
- Self-improving harness studies
  - Keep only changes that do not regress
  - Keep the evaluator outside what is changed

Complexity creep

How it happens

- Every fix adds a mechanism
- Rules, hooks and scripts overlap
  - Until nobody can hold them in their head
  - A change in one place breaks another

What to do

- Merge rules and hooks that cover the same failure
- Retire mechanisms that compensated for older models
  - They can waste tokens or make a newer one worse
- Measure each mechanism
  - How often it fires and blocks correct work
  - The time and context it uses
- Prefer a few strong mechanisms over many weak ones

A possible first week

01
One CLAUDE.md
Rules and project knowledge

→

02
One skill
For the prompt you type most

→

03
One hook
For the rule an agent broke

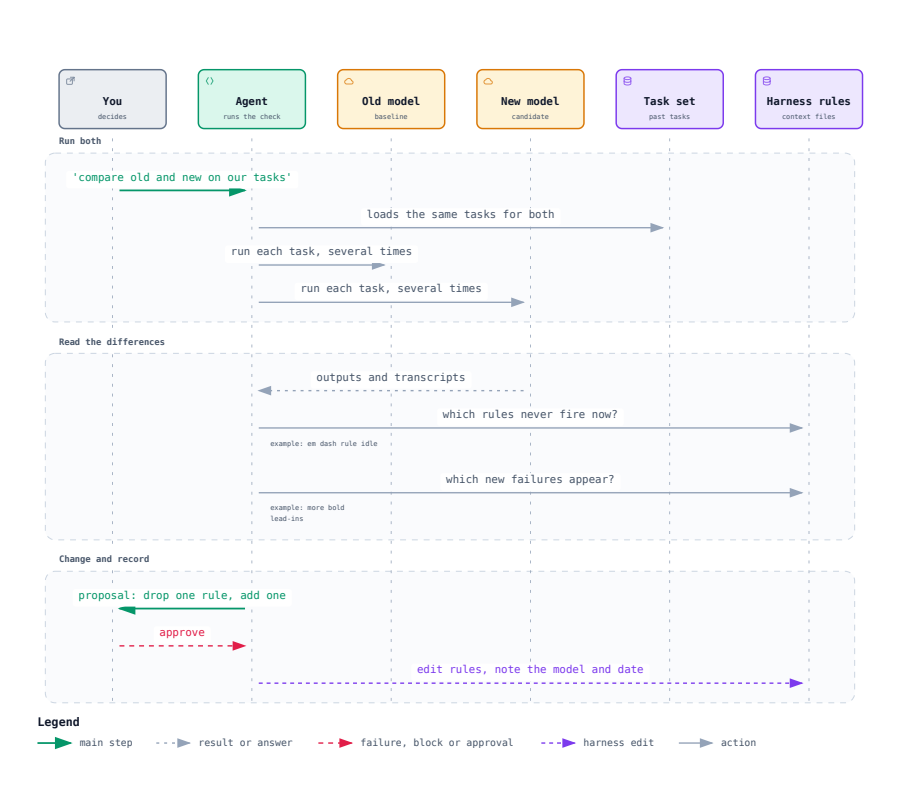
→

04
One approval rule
What needs your approval

- Each is small
  - Each one shows you the next piece of friction to work on

Model changes and trends

New models change what a harness needs.



- Re-test rules when the model changes
  - Instructions for an older model can waste tokens or hurt results
- Remove what no longer helps
  - My writing rules changed when the writing tells changed
- Setups start to move between tools
  - Agent Skills is a shared standard
  - Codex can import a Claude Code setup

Taste and spiky capabilities

- Capabilities are spiky
  - Superhuman at some tasks, weak at the next one over
- Taste is one of the weak spots
  - Code design, decisions, visuals, prose, structure
- This deck is the example
  - Bar chart for a process, walls of prose, raw URLs
  - Caught by a person, then encoded in the harness
- My view: the person steering supplies the taste
  - So replacing people soon is overstated

38%

→ 0

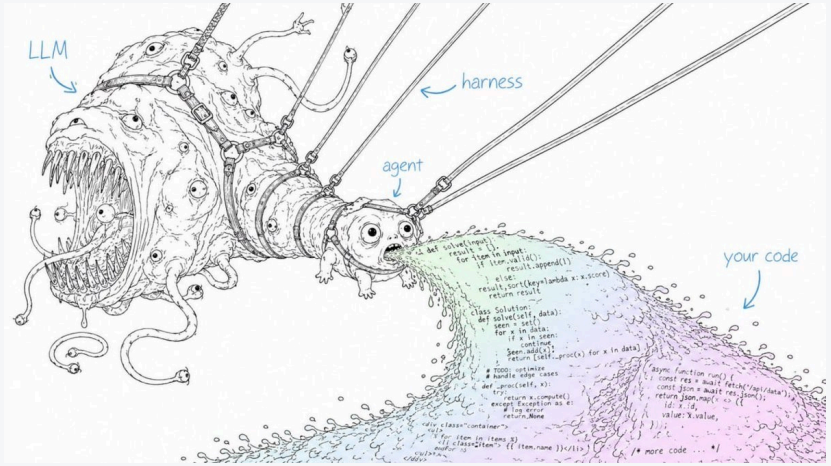
of agent pull requests passed the tests, and were mergeable as-is

Data: METR, 2025

Questions and further reading

Recap of the conductor model: you direct the agents and adjust the harness so they need less direction

- Write one context file this week
- Turn the rule an agent has broken twice into a hook
- Read the pages on the site



Appendix

0-3	4-7	8-11	12-15	16-19
Introduction	Context files	Delegation	Testing	Logging and evaluation
Agentic AI fundamentals	Skills and plugins	Session coordination	Code review	Building a harness
What is a harness?	Hooks and scripts	Orchestration	Human gates	Trends
Claude Code and Codex	Templates and stubs	Context window	Browser automation	Desktop, Web & Mobile Apps
Advanced topics				

Introduction

- From engineers running coding agents to people who use a chat app
- Applies to personal projects as well as to a job
- Advanced setups: agents coordinate, and you step in on decisions

Three depths

01 Talk slide

02 Overview page

03 Topic pages

Agentic AI fundamentals

An agent works toward a goal over many steps.

- Tools let it act: files, commands, search, services
- The context window is all it sees on one turn, up to one million tokens
- No memory between sessions unless written to files

What is a harness?

> “harness has emerged as a shorthand to mean everything in an AI agent except the model itself”
>
> Birgitta Böckeler, Thoughtworks, 2026

- Guides steer the agent before it acts
- Sensors give it feedback after it acts
- Some parts are instructions, others code
- Each loaded part costs tokens every turn

Claude Code and Codex basics

NEEDS A CLAUDE.AI SIGN-IN
Remote Control
Claude in Chrome
Computer use

Codex

- Hooks, skills and TOML sub-agents
- Goal, compact and memories commands; loop not available
- `/import` reads a Claude Code or Cursor setup

Context files

- Imports with `@path`, up to four levels deep
- Codex reads one `AGENTS.md` per directory, up to 32 KiB
- Claude Code reads `AGENTS.md` only when no `CLAUDE.md` exists
- Keep each `CLAUDE.md` under 200 lines

`.claude/rules/tests.md`

```
---
paths:
  - "tests/**"
---
Rules for test files
```

Skills, commands and plugins

FRONTMATTER	WHO CAN START IT
Default	You and the model
disable-model-invocation: true	Only you
user-invocable: false	Only the model
Codex: allow_implicit_invocation: false	Only you

- Plugins bundle skills, hooks, agents, MCP servers
- Up to six skills in one message
- Delete skills nobody invokes

Hooks, scripts and linters

> “stop, don’t do this”
>
> Claude Code hooks reference, on exit code 2

01 Context rule

02 Path rule

03 Skill

04 Hook or guard

05 Test or CI gate

Templates and stubs

• Sub-agent brief stub

```
Objective
Inputs
Output format, length cap
Tools and scope
Boundaries
When stuck
```

- A stub is a fill-in-the-blanks brief
- A template is the full shape of one kind of document
- The brief carries the safety rules word for word

Agent profiles, sub-agents and delegation

TASK	MODEL
Decisions, synthesis	Strongest
Bulk reading, edits	Cheaper

3

levels of sub-agent nesting by default in Claude Code; a setting changes it

Coordinating several sessions

- One session per machine can own installs and pushes
- Check each channel can reach its target before use

> Message order in my harness
>
> 01 SendMessage
>
> 02 Agent Mail
>
> 03 Buzz
>
> 04 A git inbox for other machines

Orchestration and workflows

FRAMEWORKS IN USE
LangGraph
OpenAI Agents SDK
CrewAI
Google Agent Development Kit
Microsoft Agent Framework

- Workflow runs: 16 agents at once, 1,000 total
- Hill-climbing keeps only measured gains

Context window and memory

Loaded again after compaction

- Root CLAUDE.md
  - Rules without paths
    - Auto memory

I compact at about 50 to 60 percent.

Others report a sweet spot below about 300k tokens (unverified).

Codex compacts near 250k tokens by default, by my observation (unverified).

Testing agent-written code

5.3%

test tampering across eight models, down from 23.6%, with a policy line and a way to report a bad test

- Run affected tests; full suite less often
- A project deleted 400,000 lines of its own tests
- Its code coverage barely moved
- Keep every gate passable wherever it runs

Code review

- Route merges through one owner or a merge queue
- Run the affected tests on the merged result
- Review briefs say that no change is a valid result

> “An agent’s approval never counts as a person’s.”
>
> Claude Code team, Solving code review at Anthropic

Human gates, decisions and alignment

Needs a person

Publishing, sending, deleting, spending money, architectural choices

The agent decides

Contained technical calls, with a record of why

When blocked

A blocked agent investigates first, and asks only if a gate still applies

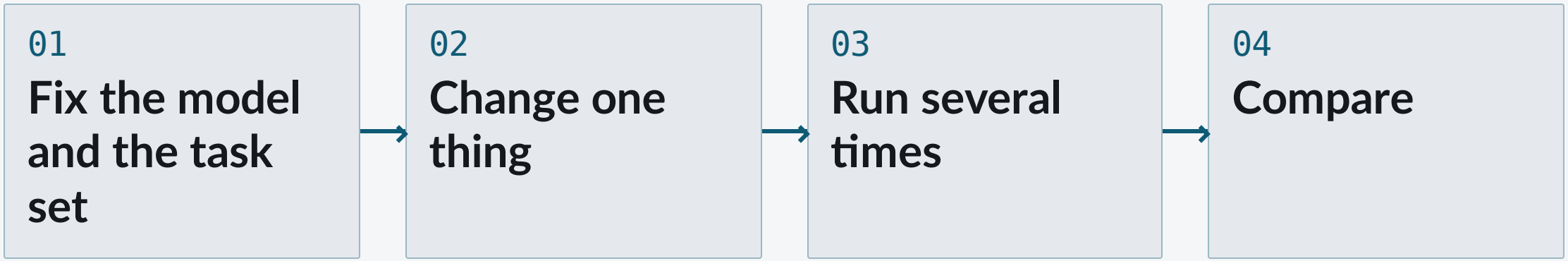
Browser automation

Treat web page content as untrusted.

ROUTE	AVAILABILITY
Claude in Chrome	Direct Anthropic plan; not through Amazon Bedrock
Computer use	Research preview on macOS, Pro, Max and Ultra
CDP or Playwright	Any provider

Logging, observability and evaluation

- Put a budget check before any job that makes paid model calls
- `claude plugin eval` compares results with and without a plugin
- About 30% of one benchmark's tasks were found broken (not yet verified)



Building your own harness

> Automatic harness evolution “fails to outperform simple test-time scaling methods”
>
> arXiv 2607.12227, 2026

- **Mine failures**
- **Make small bounded edits**
- **Keep only changes that do not regress**

Trends and model changes

> “The bottleneck, now, is all of the developer infrastructure built for human scale”
>
> Anthropic, 2026

30,000

and more code changes merged per day at Anthropic as of September 2026, up from about 2,500 a day in May

Desktop, Web & Mobile Apps

TAB OR APP	HOW HARNESS CONCEPTS MAP
Code	Claude Code itself: context files, skills, hooks, sub-agents
Projects	Project instructions and files give standing context
Cowork	Longer tasks; memory carries across chat and Cowork
Chat	Every rule is an instruction; no hooks run
Mobile	Remote Control and Dispatch reach a session on your machine

Claude web and desktop also show MCP Apps from MCP servers

Advanced topics

- Mods and function hooks: functions the client loads and calls on tool calls, prompts and turns
- MCP Apps in the desktop and web apps: an MCP tool returns an interactive view in Claude web and desktop
- Taste and spiky capabilities: where models are strong, where they are weak, and why it matters
- On the site: mods and function hooks, MCP Apps, taste and spiky capabilities

Mods and function hooks

A function hook is a function the client loads and calls on tool calls, prompts and turns. A mod is a plugin whose hooks module registers them.

Shell hook

- A shell command or script
  - A new process per event
- Cannot draw
  - Exit code and output only
- Typical use: guards and checks

Function hook

- JavaScript or TypeScript
  - Functions inside the client
- Guards or changes behaviour
  - Like a shell hook, without a process per event
- Can also change the interface
  - Panes, status line, toasts, slash commands
  - A common use, not a requirement

MCP Apps in the desktop and web apps

01
A tool runs
On an MCP server

→

02
It returns a view
An interactive interface, not only text

→

03
The client shows it
In a sandboxed frame in the chat

→

04
You use it
The view can call the server's tools

- The chat-app counterpart to interface mods
  - Provided by a server, not loaded into the client
  - Sandboxed and limited to its own panel
- Claude web and desktop render them

Remote Control and mobile

How it works

- `/remote-control` (alias `/rc`) links a local session
  - To Claude Code on the web and the iOS and Android apps
- Claude keeps running on your machine
  - Files, MCP servers and tools stay local
- Push notifications on your phone
  - When a task finishes or needs a decision

Availability

- Pro, Max, Ultra, Team and Enterprise plans
- Needs a claude.ai sign-in
  - API keys are not supported
- Not on Bedrock, Vertex, Foundry or a gateway
- Team and Enterprise: an Owner turns it on
  - In Claude Code admin settings

Taste in practice

Spiky capabilities

- Inside the frontier: 40% higher quality; outside it, accuracy fell from 84% to 60 to 70% (jagged frontier study)
- Task length agents finish doubles every few months (METR time horizons), on well-specified tasks
- Experienced developers took 19% longer with AI, while feeling faster (METR study)

Supplying the taste

- Write down the part of taste that can be written down
  - Style guides, writing checks, link checks, layout rules
- Review for mergeable, not only passing
  - Design, clarity and fit, beyond the tests
- Date the claim
  - The frontier moves; this is how it looks today